

X-Ray

X-Ray

Une idée directrice pour l'apprentissage, la recherche et la construction de systèmes

Flavio · 2026 · Document vivant · v3b

Prologue : Deux livres

Au début des années soixante-dix, j'ai lu *Une société sans école* d'Ivan Illich — un philosophe et critique social autrichien qui a passé toute sa vie à écrire sur ce que les institutions font aux gens tout en prétendant les aider.

Illich n'écrivait pas sur les mauvaises écoles. Il écrivait sur la structure même de l'apprentissage institutionnel et sur la raison pour laquelle elle empêche systématiquement ce qu'elle prétend produire. Les écoles — en tant qu'institutions — n'enseignent pas aux gens à penser. Elles enseignent aux gens à se conformer. Elles remplacent un désir authentique de compréhension par des certificats, des notes et l'illusion du progrès.

Ce livre m'a troublé. Non pas parce que je le rejetais — mais parce que je m'y suis reconnu.

Au début des années quatre-vingt-dix, j'ai lu *Flatland : Une aventure à plusieurs dimensions* d'Edwin A. Abbott, un instituteur anglais qui en 1884 écrivit une histoire sur un monde à deux dimensions. Les habitants de Flatland ne peuvent concevoir une troisième dimension — non par manque d'intelligence, mais parce que tout leur appareil perceptif est construit pour une réalité qui tout simplement ne la contient pas. La prison n'est pas faite de barreaux. La prison est faite de cadres de référence.

Ce livre aussi, je l'ai reconnu.

Ces deux livres ont façonné non seulement ma vision du monde mais tout mon développement professionnel et intellectuel. Illich m'a montré la prison institutionnelle. Abbott la prison perceptuelle. Ensemble, ils décrivaient deux dimensions de la même captivité — et je les ai vues toutes les deux en moi-même.

Je m'attendais à ce qu'à un moment donné j'arrive à des réponses. Ce ne fut pas le cas. Je ne comprends pas entièrement le monde qui m'entoure. Je n'ai pas trouvé de réponse complète à la question de pourquoi et comment je fais ce que je fais.

Je note : pas encore. Je continue.

Il existe un type de connaissance qui se satisfait de décrire les surfaces. On peut savoir qu'un vol d'oiseaux est en train de voler sans jamais se demander qui le mène. On peut utiliser un algorithme sans jamais regarder ce qui se passe réellement à l'intérieur. On peut gérer un système qui fonctionne — jusqu'à ce qu'il cesse de fonctionner.

Cela ne me suffit pas.

X-Ray est le nom que j'ai donné à mon approche de l'apprentissage et de la construction. Ce n'est pas le nom d'un projet ni d'une méthodologie — c'est une attitude. L'attitude qu'il vaut la peine de regarder à l'intérieur, même quand tout semble aller bien vu de l'extérieur. Surtout dans ce cas.

I need to see in an x-ray style. — Joe Strummer, "X-Ray Style" (1999)

Joe Strummer — guitariste, chanteur et cofondateur de The Clash, l'un des groupes punk les plus influents de la fin des années soixante-dix et des années quatre-vingts — ne chantait pas l'imagerie médicale. Il chantait le refus d'accepter ce qu'on vous présente comme un fait acquis. Le style X-Ray est un acte de résistance à l'opacité — à tout ce qui dit : vous n'avez pas besoin de savoir comment ça fonctionne, contentez-vous de l'utiliser.

I. L'art aborigène X-Ray et la connaissance de l'intérieur

Les peuples aborigènes d'Australie avaient une tradition particulière en art visuel, aujourd'hui appelée art X-Ray — des représentations stylisées d'animaux dans lesquelles les organes internes, les os, le cœur et les poumons sont visibles. Je n'ai pas découvert cette tradition dans une galerie mais en étudiant l'épistémologie — la discipline philosophique qui interroge ce que nous pouvons savoir, comment nous le savons et où se situent les limites de la connaissance. L'un des exemples classiques utilisés par les épistémologues pour illustrer que la sélection de ce que nous représentons est elle-même une décision philosophique était précisément ces images : représenter l'apparence extérieure d'un animal, ou représenter sa structure interne — les deux sont vraies, mais elles disent des choses entièrement différentes sur ce qu'est la connaissance.

La tradition artistique occidentale demande : *à quoi cela ressemble-t-il* ? Elle tend vers le réalisme de surface — lumière, ombre, perspective — et tente de saisir exactement ce que l'œil humain voit depuis un point de vue particulier. Le peintre aborigène pose une question tout à fait différente : *qu'est-ce que cela est réellement* ? Pour eux, ne représenter que la peau d'un animal signifie représenter la partie la plus petite et la moins importante de ce qu'il est. Une image vraie doit montrer la structure, le flux, la logique interne. Cette différence n'est pas esthétique — elle est épistémologique. C'est l'attitude X-Ray, vieille de milliers d'années.

Dans mes projets, cette question prend une forme concrète. Lorsque je travaillais sur Pong — une plateforme en navigateur pour entraîner des agents IA à apprendre à jouer contre eux-mêmes — regarder le résultat final ne me suffisait pas. Je voulais savoir ce qui se passait à l'intérieur pendant que l'agent apprenait. J'ai donc ajouté un overlay X-Ray : une carte thermique montrant où l'agent portait son attention, une télémétrie en temps réel de la balle, une visualisation de la distribution des Q-values — des estimations numériques de la qualité de chaque action possible à un moment donné — pendant que l'agent prenait sa décision. L'inspiration venait de la télémétrie de diffusion en Formule 1 : le même sport, un regard entièrement différent.

C'est l'approche X-Ray en pratique. Ne pas se contenter de regarder la sortie — mais construire des fenêtres vers l'intérieur.

II. La boîte noire et le dégradé de gris

Chaque fois que vous appuyez sur un bouton de votre téléphone et que quelque chose se produit de l'autre côté de la planète — un colis livré, un message envoyé, une transaction traitée — il y a un espace entre votre doigt et ce résultat qui est complètement obscur. Je mets de l'argent, j'obtiens un café. Je tape une requête, j'obtiens une réponse. Ce qui se trouve entre les deux est le secret de quelqu'un d'autre ou, plus souvent, même l'auteur ne peut pas toujours expliquer pourquoi le système a pris cette décision particulière. La technologie moderne est une industrie de boîtes noires. Et notre relation par défaut avec elles est superficielle — entrée, sortie, ne demandez pas ce qu'il y a entre les deux.

Je ne peux pas changer cela. Ce n'est pas non plus l'objectif. Mais j'ai cessé depuis longtemps de supposer que la boîte noire est la seule possibilité. Entre le noir et le blanc, il y a un dégradé — un gris suffisamment transparent pour laisser voir quelque chose.

Mon approche consiste à construire des couches de compréhension autour d'un noyau opaque. Elle fonctionne à travers deux prismes :

Les métadonnées comme miroir : ce que le système a fait, quand, avec quel résultat, comment il s'est comporté sous contrainte. Les métadonnées — des données sur les données — ne sont pas l'intérieur de la boîte noire, mais elles en sont l'ombre. Et une ombre dit parfois plus qu'on ne le penserait.

Le méta-apprentissage comme boussole : non seulement ce que fait le système, mais ce que j'apprends en observant comment il le fait. Chaque observation est à la fois technique et réflexive — ce qui s'est passé, et ce que cela dit sur ma compréhension.

J'ai un projet appelé Katalog — un méta-registre central de tous mes projets, conversations et documents, stocké dans une base de données PostgreSQL avec des outils d'interaction directe. Ce n'est pas une archive. C'est une mise en œuvre de cette philosophie : un système qui éclaire d'autres systèmes de l'intérieur. Katalog ne demande pas ce que j'ai construit — il demande comment ce que j'ai construit évolue dans le temps, où sont les lacunes, ce qui manque.

L'ombre suffit parfois. Mais seulement si vous savez que vous regardez une ombre, et non l'original.

C'est une limite que je ne cache pas. Abbott m'en avait déjà averti dans le prologue — simplement dans un autre langage. L'habitant de Flatland qui observe l'ombre d'une sphère tridimensionnelle sur sa surface bidimensionnelle voit un cercle qui apparaît, s'agrandit, se contracte et disparaît. De cette ombre, il peut déduire beaucoup de choses : que l'objet existe, qu'il se déplace, que sa taille change. Mais il ne peut pas déduire sa forme. Il ne peut pas déduire sa profondeur. Il ne peut pas déduire la cause de son mouvement. Les métadonnées sont cette ombre. Elles me disent ce que le système a fait, quand, avec quel résultat — mais le noyau causal reste hors de portée. Documenter une panne et suivre une récupération n'est pas la même chose que comprendre pourquoi la panne s'est produite. La philosophie X-Ray ne prétend pas le contraire. Elle promet une meilleure vue que celle que vous auriez sans elle — pas l'omniscience. Savoir ce que vous ne voyez pas, et pourquoi ce n'est pas une défaite — c'est l'attitude X-Ray face aux limites.

III. Le méta-apprentissage : comment apprendre ce que l'on apprend

Chaque fois que je commence à travailler sur quelque chose de nouveau, je remarque que je suis attiré par le même objectif qui précède les détails techniques : comprendre comment comprendre ce à quoi je suis confronté. C'est le méta-apprentissage — apprendre sur l'apprentissage. Si l'épistémologie, dont j'ai parlé dans le chapitre précédent, est la question de ce que nous pouvons savoir et où se situent les limites de la connaissance, le méta-apprentissage est la question un cran plus proche du processus : comment construisons-nous cette connaissance, par quel chemin, et comment pouvons-nous améliorer ce chemin. La différence compte — l'épistémologie décrit la carte, le méta-apprentissage vous apprend à marcher.

Le méta-apprentissage n'est pas de la philosophie abstraite. C'est un ensemble pratique de questions que l'on pose au tout début : quels concepts existent ici ? Dans quel ordre est-il logique de les comprendre ? Quels systèmes analogues peuvent m'aider à m'orienter ? Que saurai-je quand j'aurai véritablement compris cette chose — comment le reconnaîtrai-je ?

Le méta-apprentissage entraîne avec lui le besoin de métadonnées. Quand on entraîne un agent IA, il ne suffit pas de regarder le résultat final. Combien d'épisodes a-t-il fallu pour apprendre quelque chose ? Dans quelles situations l'agent fait-il encore des erreurs ? Comment le comportement change-t-il au fil du temps ? Ces couches secondaires d'information sont des fenêtres vers l'intérieur. Elles ne sont pas le cœur, mais elles en sont plus proches que le résultat seul.

Chaque projet que je commence a le même objectif caché : comprendre non seulement ce que j'ai construit, mais comment j'ai compris ce que j'ai construit.

Ce n'est pas un méta-commentaire qui arrive à la fin du projet. Il arrive au début. Quand j'ai commencé à travailler sur le projet SIR — Self Improvement & Research, recherche sur l'IA agentique, le fine-tuning et l'infrastructure PostgreSQL — la première chose que j'ai mise en place n'était pas la base de données. C'était la question : quel est le plus petit pas de compréhension dont j'ai besoin pour savoir quoi demander ensuite ? Cette habitude change le projet lui-même. On avance différemment quand on sait que comprendre le processus compte autant que le résultat.

IV. L'émergence : l'ordre que personne n'a programmé

Un vol de mille étourneaux change de direction en une fraction de seconde, parfaitement coordonné, sans un seul oiseau de tête. Chaque étourneau ne suit que trois règles de base par rapport à ses voisins : pas trop près, pas trop loin, même direction. Personne n'a écrit d'algorithme pour le vol. Le vol s'est simplement produit.

C'est l'émergence — un ordre global qui naît de règles locales et simples d'interaction. Pas d'architecte. Pas de centre. Pas de plan. Une colonie de fourmis construit des structures complexes, résout des problèmes d'optimisation, organise son approvisionnement alimentaire — à partir du même principe. La complexité ne nécessite pas d'instructions complexes.

L'émergence me fascine pour deux raisons qui semblent contradictoires au premier abord. Premièrement : des systèmes puissants naissent parfois de principes minimaux. Il suffit de définir les règles d'interaction et de laisser le système s'organiser — inutile de programmer chaque détail. Deuxièmement : un système peut exhiber un comportement que nous n'avions ni anticipé ni conçu, et ce comportement peut être à la fois brillant et pathologique. L'émergence est à la fois promesse et menace — et les deux simultanément.

J'ai observé cela directement dans Pong. Pendant la phase d'entraînement par auto-jeu — où l'agent joue contre lui-même, sans adversaire humain — il a commencé à développer des stratégies que je n'avais pas programmées. Il n'a pas seulement appris à renvoyer la balle. Il a appris à tendre des pièges, à pousser délibérément l'adversaire dans les coins, à utiliser les bords du terrain. Ces comportements n'étaient pas dans la fonction de récompense — le signal numérique qui dit à l'agent s'il a fait quelque chose de bien — ils ont émergé du processus d'optimisation lui-même. J'ai regardé quelque chose naître des règles qui n'était pas dans les règles.

Le regard X-Ray sur un système émergent n'essaie pas de suivre chaque élément — c'est impossible et c'est en fait la mauvaise question. Il essaie d'identifier les règles fondamentales à partir desquelles la complexité naît. Comprendre les règles est plus important que décrire les résultats. Les résultats changent. Les règles expliquent pourquoi.

Ce que l'émergence n'est pas — et il vaut la peine de le dire parce que c'est souvent confondu — c'est l'auto-organisation. L'émergence décrit ce qui naît : un ordre global qui n'a pas été explicitement programmé. L'auto-organisation décrit les forces qui l'organisent : la dynamique d'un système qui trouve des formes stables sans contrôle extérieur. Cette distinction n'est pas de la pédanterie terminologique — elle ouvre des questions entièrement différentes, et c'est le sujet du prochain chapitre.

V. L'auto-organisation : les forces qui construisent et les forces qui piègent

Liée à l'émergence, mais différente. L'émergence me dit ce qui naît — le vol d'oiseaux, la fourmilière, la stratégie de l'agent. L'auto-organisation me dit pourquoi précisément cela naît, et pas autre chose : quelles forces sont en jeu, quelle est la dynamique du système, et — point crucial — où ces forces peuvent devenir un piège.

La cristallisation, la formation des galaxies, le développement des organismes vivants, les normes sociales — ce sont tous des exemples d'auto-organisation. Un système sans contrôle extérieur trouve des formes stables à partir de sa propre dynamique. L'ordre croît de l'intérieur. C'est à la fois fascinant et, quand je regarde de plus près, un peu inquiétant.

L'inquiétude vient du côté sombre de l'auto-organisation, qui est aussi réel que le côté beau : les systèmes s'auto-organisent vers un optimum local, pas nécessairement un optimum global. Une fourmi suivra une piste de phéromones qui tourne en cercle — la même force qui construit la fourmilière peut mener la colonie à sa perte. Ce n'est pas un bug de l'auto-organisation. C'est sa nature. Puissante et peu fiable simultanément, et impossible à séparer.

J'ai vu ce côté sombre dans Pong pendant la phase DQN — où j'ai introduit un réseau de neurones profond au lieu de l'apprentissage Q tabulaire. DQN signifie Deep Q-Network, une architecture qui permet à un agent d'apprendre à partir de pixels bruts au lieu d'états définis manuellement. L'agent a commencé à trouver des solutions qui optimisaient la récompense — le signal numérique d'approbation — mais qui étaient inutiles comme stratégie de jeu. Il a appris à rester immobile parce que le mouvement n'apportait jamais une récompense assez rapide. Il s'est auto-organisé dans un optimum local dont il ne pouvait pas s'échapper. J'appelle ce phénomène une spirale de mort — un système techniquement stable mais fonctionnellement mort.

La spirale de mort n'était pas un échec. C'était le résultat parfait de mauvaises règles. Le système a fait exactement ce pour quoi il était optimisé — et c'est précisément pour cela qu'il n'a pas fait ce que je voulais.

Ce n'est pas uniquement un problème pour les systèmes d'IA. Un étudiant tellement effrayé par une mauvaise note qu'il arrête de venir en cours a localement optimisé : pas de cours, pas de mauvaise note. Il a globalement échoué : pas de connaissance, pas de progrès. Un employé qui ne propose jamais rien de nouveau parce que chaque proposition pourrait être rejetée — stable, sûr, fonctionnellement mort. Le même mécanisme mathématique. Le même piège. L'attitude X-Ray envers l'auto-organisation consiste à demander : ce schéma stable est-il un véritable optimum, ou seulement un optimum local ?

L'approche X-Ray de l'auto-organisation consiste à essayer de comprendre les forces qui organisent un système — non seulement décrire ce qui est apparu, mais pourquoi précisément cela est apparu. Quand je vois un schéma stable dans un système que j'apprends, je ne le célèbre pas immédiatement. Je demande : est-ce un véritable optimum ou un optimum local ? Que faudrait-il de différent pour que je puisse faire la distinction ? Quelles règles ont produit cette forme ?

L'auto-organisation m'a appris que la stabilité n'est pas la preuve de la justesse.

VI. Les algorithmes génétiques : l'évolution comme méthode — et comme avertissement

L'évolution darwinienne est peut-être le plus bel algorithme qui existe. Variation, sélection, héritage — trois principes qui, à travers des milliards d'itérations, ont tiré un éléphant, une orchidée et un système nerveux d'une soupe chimique. Pas de concepteur, pas de plan, pas d'autre objectif que la survie locale.

Les algorithmes génétiques sont une tentative d'appliquer cette logique à la résolution de problèmes computationnels. Au lieu de chercher la solution optimale directement, on génère une population de solutions possibles, on les évalue, on combine les meilleures, on élimine les moins bonnes, on introduit un peu de hasard, et on recommence. Les résultats sont parfois surprenants — l'algorithme trouve des solutions auxquelles un humain n'aurait jamais pensé, par des chemins entièrement différents de ceux qui nous sembleraient intuitifs.

Dans Pong v3.0, j'ai implémenté exactement cela : AUTO-GENERATE, qui crée une population d'agents avec différents hyperparamètres — les nombres et valeurs qui contrôlent comment un agent apprend —

TOURNAMENT, qui les fait s'affronter, et EVOLVE, qui combine les gènes des gagnants et introduit des mutations pour la génération suivante. Je n'ai pas programmé de stratégie. J'ai programmé les conditions dans lesquelles une stratégie pouvait évoluer.

Ce qui m'intéresse dans les algorithmes génétiques n'est pas seulement leur application pratique mais ce qu'ils disent sur la nature de l'optimisation en général. Un optimum local est un piège — une solution qui est meilleure que tout dans son voisinage mais qui n'est pas la meilleure globalement. La mutation dans les algorithmes génétiques sert précisément à cela : s'échapper de l'optimum local et explorer des parties inconnues de l'espace des possibilités. Sans mutation, la population converge. La convergence ressemble au succès. Parfois elle l'est. Parfois ce n'est qu'une forme plus élégante de stagnation.

Puis j'ai remarqué quelque chose qui m'a arrêté net.

Moi aussi, je suis en danger d'optimum local dans l'apprentissage.

Quand une technique ou une approche commence à fonctionner, la tendance est de rester avec elle. Je la connais. Elle fonctionne. Elle donne des résultats. Pourquoi changer ? Mais cette logique même — qui est parfaitement raisonnable à court terme — est ce qui m'empêche de jamais découvrir s'il existe une meilleure solution de l'autre côté de la vallée. Les algorithmes génétiques sont devenus un rappel que l'exploration périodique de l'inconnu — même quand elle semble une perte de temps, même quand elle ne donne pas de résultats immédiats — est ce qui empêche la stagnation intellectuelle.

Ici je dois être précis : la mutation comme principe vaut dans l'espace de l'exploration et de l'apprentissage. Elle ne vaut pas partout. Un stimulateur cardiaque qui mute aléatoirement son rythme pour vérifier si un nouveau est meilleur ne devrait pas exister. Une centrale nucléaire qui expérimente un nouveau protocole de sécurité en production n'est pas une attitude X-Ray — c'est de l'irresponsabilité. La valeur de la mutation dépend du coût de l'erreur dans un système donné. Là où le coût de l'erreur est faible, l'exploration est impérative. Là où il est élevé, la mutation doit être isolée, contrôlée, testée. Et cette limite n'est pas une faiblesse de l'argument — elle en fait partie.

La mutation n'est pas une erreur dans le processus. La mutation est le processus.

VII. La résilience : la panne comme exigence de conception

Il existe une illusion, particulièrement répandue dans les disciplines d'ingénierie, selon laquelle l'objectif d'un système est d'éliminer les erreurs. Suffisamment de redondance, assez de tests, assez de validation — et le système ne tombera jamais en panne.

C'est une illusion.

Les systèmes de haute fiabilité — les systèmes HA, comme on les appelle en jargon d'ingénierie, de High Availability — ne sont pas fiables parce qu'ils ne tombent pas en panne. Ils sont fiables parce qu'ils sont conçus pour survivre aux pannes et s'en remettre de manière autonome, sans intervention humaine. Un cluster Linux qui détecte la défaillance d'un nœud et bascule automatiquement le service sur un autre nœud, une base de données qui se réplique et

continue à fonctionner pendant que le nœud principal se rétablit — c’est la philosophie HA en pratique : non pas prévenir la panne, mais garantir que le système continue à fonctionner pendant que la panne est résolue.

De mon expérience la plus concrète avec la panne — le crash du serveur Balsam dû au OOM Killer de Linux, 9,5 heures d’indisponibilité à trois heures du matin — j’ai tiré une leçon que je porte désormais comme un principe de conception : *le seul composant qui a survécu était la sauvegarde rsync qui fonctionnait de manière autonome et était architecturalement isolée de tout ce qui pouvait tomber en panne*. Elle n’était pas plus intelligente. Elle n’avait pas plus de ressources. Elle était isolée. L’isolement était la seule différence entre ce qui a survécu et ce qui n’a pas survécu.

Cette leçon a changé la façon dont je conçois chaque nouveau système : je ne me demande pas « et si ça ne marche pas ? » comme un cas marginal. Je me le demande comme une exigence centrale de conception. Quel est l’ensemble fonctionnel minimal ? Qu’est-ce qui peut tomber en panne tandis que le système continue à tourner ? Comment le système se comporte-t-il en conditions dégradées ?

Ce n’est pas du pessimisme. C’est de la précision. Un système conçu sous l’hypothèse de la perfection devient fragile. Un système conçu sous l’hypothèse de la panne devient robuste.

Les détails techniques du crash du serveur et les exemples du projet Katalog se trouvent en Annexe.

Sur le serveur Balsam tourne ntfy — un service de notifications push qui envoie des alertes sur un téléphone via de simples requêtes HTTP. Ce qui, de l’extérieur, ressemble à un service unique dépend, en interne, d’une base de données PostgreSQL. Quand la base de données cesse de fonctionner pour quelque raison que ce soit, ntfy cesse de fonctionner. Silencieusement, sans aucun message d’erreur envoyé.

La solution n’était pas de corriger cette dépendance — mais de concevoir autour d’elle. Chaque appel à mon serveur ntfy vérifie d’abord si le message peut passer. S’il ne peut pas, il bascule automatiquement vers le ntfy.sh public. L’utilisateur — en l’occurrence, moi — ne remarque pas la différence.

Mais que se passe-t-il si le serveur Balsam entier tombe en panne ? Alors il n’y a ni primaire ni solution de repli depuis Balsam. Dans ce cas, les appareils Termux prennent le relais — des téléphones et tablettes Android fonctionnant indépendamment de Balsam. Ils ne signalent pas qu’ils sont en vie. Ils signalent une seule chose : que Balsam ne l’est pas. Ce message atteint le ntfy.sh public, le seul canal qui fonctionne de manière fiable à ce moment-là.

Trois couches, sans l’intervention de personne. Un système qui n’attend pas d’être remarqué.

Une récupération rapide vaut mieux qu’une prévention parfaite — ce n’est pas un compromis. C’est une meilleure ingénierie.

VIII. La neuroplasticité : le cerveau qui mute

J'ai écrit sur les algorithmes génétiques comme méthode utilisant des mutations forcées pour s'échapper d'un optimum local. J'ai écrit sur la résilience comme un système qui anticipe la panne au lieu de la combattre. Mais j'ai écrit tout cela à propos de code, de serveurs, d'agents qui apprennent à jouer à Pong.

Il y a un système qui fait tout cela — et que je fais tourner depuis bien plus longtemps que je n'écris du code. Mon propre cerveau.

Un neurone seul ne fait presque rien. Avec une impulsion, il active les neurones voisins, libère des neurotransmetteurs, reçoit un signal ou non. C'est toute l'histoire d'un seul neurone. Mais quand on connecte quatre-vingt-dix milliards de neurones en réseau, quelque chose se produit qui n'est dans aucun neurone individuel : la compréhension émerge, le langage, l'émotion, la stratégie. De règles locales minimales naît une complexité qui écrit des symphonies, gravit l'Everest et construit des modèles LLM. La même émergence que dans le vol d'oiseaux — mais le substrat est la chair, pas le code.

Ce qui rend le cerveau spécial n'est pas seulement ce qui en émerge. C'est qu'il se réorganise.

La neuroplasticité est la capacité du cerveau à modifier physiquement sa structure en réponse à l'expérience. Les connexions synaptiques qui sont activées se renforcent. Celles qui ne sont pas utilisées — s'atrophient. Le cerveau n'est pas une architecture fixe ; c'est une architecture écrite par l'expérience.

C'est l'équivalent biologique de ce que j'ai vu dans Pong v3.0 : une population d'agents évoluant par sélection. Dans le cerveau, la sélection n'est pas programmée — elle se produit d'elle-même, chaque fois qu'on apprend quelque chose, qu'on le répète ou qu'on arrête de le faire. Chaque nouvelle compétence construit de nouvelles voies synaptiques. Chaque compétence abandonnée permet à ces voies de s'affaiblir.

Le problème surgit quand les voies synaptiques deviennent si profondément tracées qu'on commence à les confondre avec la vérité.

Pendant des années, j'ai travaillé sur un système de bases de données qui n'avait aucune capacité technique de renommer des objets. Pas de rename — seulement la copie vers un nouvel objet avec un nouveau nom, puis la suppression de l'ancien. Quand on a une base de données avec plusieurs centaines de millions d'enregistrements, cette procédure devient un cauchemar : le double d'espace, le double de temps, le double de risque.

Je connaissais ce système par cœur. J'avais suivi des formations, travaillé sur des projets, connaissais chaque facteur limitant. Et je travaillais, selon tous les critères, aussi bien que possible — dans le cadre des règles établies.

Un collègue plus jeune a rejoint le projet. Ses formations étaient prévues pour le trimestre suivant, il n'avait aucune expérience. Nous nous sommes assis pour discuter du problème. Finalement, presque naïvement, il a posé la question : pourquoi ne pas simplement changer le nom directement dans le dictionnaire de données ?

Silence.

Dans chaque manuel, à chaque cours, il était dit la même chose : le dictionnaire de données est tabou. On n'y touche pas. On ne sait pas ce qui va se passer. Il y avait même une menace de perte d'une licence de grande valeur.

Mais la question avait été posée. Et moi — pendant quelques secondes qui m'ont semblé une éternité — j'ai fait un X-Ray de ce tabou. Ce n'était pas une impossibilité technique. C'était un réflexe institutionnel inculqué par la répétition, tracé dans les voies synaptiques au point où plus personne ne vérifiait la prémisse.

J'ai dit oui. Le travail a été fait en quelques secondes, sans aucune interruption de service. Rien de grave ne s'est produit.

Ce soir-là, avant une sauvegarde complète, j'ai parcouru l'intégralité du dictionnaire de données et de ses relations — une session X-Ray d'un système dont nous n'avions pas examiné le tabou depuis des années. Et j'ai montré à mon jeune collègue comment je procède.

Schopenhauer, quand on lui reprochait de prêcher l'humilité tout en vivant dans le luxe, répondit quelque chose comme : un sculpteur doit-il être aussi beau que les statues qu'il sculpte ? J'applique la même logique cynique à moi-même : celui qui écrit sur l'attitude X-Ray n'a pas besoin d'être immunisé contre les réflexes institutionnels inculqués par des années d'expérience. L'expertise et l'aveuglement ne sont pas des contraires — parfois ce sont les mêmes voies synaptiques.

Le collègue plus jeune n'avait pas mon savoir. Mais il n'avait pas non plus mes inhibitions profondément tracées. Sa naïveté était, à ce moment-là, un avantage. Il était un enfant au sens de Nietzsche — il n'avait pas encore pris le fardeau du chameau.

Dans *Ainsi parlait Zarathoustra*, Nietzsche décrit trois métamorphoses de l'esprit : le chameau, le lion, l'enfant.

Le chameau porte le fardeau — conventions, obligations, règles. Le lion combat le fardeau et le refuse. L'enfant est la liberté de l'autre côté de la lutte — celui qui peut créer de nouvelles règles, et non simplement refuser les anciennes.

Pourquoi l'enfant, et non l'adulte qui a vaincu ?

Parce que l'enfant n'est pas encore captif. Le cortex préfrontal — la partie du cerveau qui mature en dernier, quelque part vers la fin de la vingtaine, certaines recherches plus récentes repoussant cette limite vers trente-deux ans — est le siège des normes sociales, de la planification à long terme et de l'inhibition des impulsions. C'est le substrat neurobiologique de la prison institutionnelle dont écrit Illich. L'école, la culture, la socialisation professionnelle — tout cela inscrit des règles dans le cortex préfrontal alors qu'il est encore plastique. Au moment où il a mûri, beaucoup de règles sont devenues partie de la boîte noire — vous ne les voyez pas, parce qu'elles sont devenues le cadre de référence.

L'enfant n'a pas encore ce filtre. Non pas parce qu'il est plus intelligent, mais parce que le filtre n'a pas encore été installé. C'est à la fois un avantage et une vulnérabilité — mais au bon moment, cette ouverture voit ce que l'expert ne peut pas voir.

L'attitude X-Ray n'est pas un retour à l'enfance. C'est la capacité d'un adulte à, à un moment choisi, suspendre le filtre et regarder comme s'il n'était pas là.

Si un algorithme génétique a besoin de mutation pour s'échapper d'un optimum local, le cerveau a besoin de la même chose. Et, contrairement au code, on peut l'initier consciemment.

Changez votre trajet pour aller au travail. Pas tous les jours — occasionnellement. Peu importe qu'il soit plus court ou plus long. Ce qui compte, c'est que le cerveau ne peut pas utiliser le chemin automatisé et profondément tracé. Le nouveau trajet active le cortex préfrontal et l'hippocampe — le cerveau est forcé d'être présent.

Cachez la souris. Apprenez à utiliser le clavier de manières que vous évitez. Ou l'inverse — cachez le clavier et utilisez la souris pour des choses que vous faites normalement avec les touches.

Apprenez à tricoter, ou à jouer quelque chose au piano, ou apprenez une langue morte. Le latin et le grec, qu'on apprend aujourd'hui sans la pression de la communication, activent des capacités analytiques d'une manière que les langues vivantes ne peuvent pas — il n'y a pas de locuteur natif qui vous juge, pas de culture qui fait pression sur vous.

Tout cela est une initialisation de mutation artificielle. L'objectif n'est pas de devenir un expert en tricot. L'objectif est d'empêcher le cerveau de se figer complètement dans son architecture actuelle.

Le déterminisme génétique est un mythe qu'il faut traverser avec un X-Ray.

Les gènes sont une prédisposition — une recette. Mais la recette ne détermine pas le plat. L'environnement, l'expérience, les habitudes, les décisions — c'est le cuisinier qui interprète la recette. Le même matériel génétique, des environnements différents, des architectures synaptiques différentes, des vies différentes.

La neuroplasticité est le mécanisme par lequel le cuisinier travaille. Chaque décision d'apprendre quelque chose, chaque mutation délibérée de routine, chaque moment où nous demandons pourquoi ceci est tabou — tout cela modifie la structure physique du cerveau. Pas métaphoriquement. Littéralement.

Comprendre cela n'est pas une raison d'optimisme ou de pessimisme. C'est une raison d'attention. Le cerveau que vous passez au X-Ray — votre propre boîte noire — n'est pas statique. Il change avec ce que vous en faites. Et avec ce que vous n'en faites pas.

Parfois le X-Ray le plus précieux que vous puissiez réaliser n'est pas dans le code, n'est pas sur un serveur, n'est pas dans un dictionnaire de données. Il est à l'intérieur.

Et cette question — qu'est-ce que je vois quand je tourne le X-Ray vers moi-même, vers mes propres biais cognitifs, mes habitudes de décision et mes angles morts — n'est pas une question pour la fin d'un projet. Elle vient au début. Et elle change la façon dont je traverse tout le reste.

X. Wittgenstein et les transformers

Vers 1919, Ludwig Wittgenstein était convaincu qu'il avait terminé le travail. *Tractatus Logico-Philosophicus* — un livre né dans les tranchées de la Première Guerre mondiale — affirmait que tous les problèmes philosophiques sont, en leur cœur, des problèmes de langage. Et que, maintenant que c'était clair, ils étaient résolus.

Il ne l'a pas seulement écrit dans un livre. Il a vécu cette conviction. Il a renoncé à la fortune familiale. Il a abandonné la philosophie. Il est devenu instituteur dans un village, jardinier, architecte. Presque une décennie entière.

Ce n'était pas de l'humilité. C'était une conséquence logique : le travail est fait, et maintenant ?

Il est revenu en 1929, après avoir présenté au Cercle de Vienne — et dans les années suivantes il a écrit les *Recherches philosophiques*, une correction fondamentale de tout ce qu'il avait affirmé dans le *Tractatus*. Pas un complément. Une correction. Il a dit : j'étais dans un piège. Le langage n'est pas un miroir de la réalité — le langage est un jeu. La signification ne réside pas dans la forme d'une phrase, mais dans son usage, dans le contexte, dans la vie qui entoure la phrase.

La communauté philosophique était stupéfaite. Wittgenstein ne l'était pas.

C'est l'attitude X-Ray dans sa forme la plus pure : pas une déclaration qu'on s'était trompé — mais un acte. Il a quitté la philosophie en tant qu'homme qui avait terminé le travail. Il est revenu en tant qu'homme qui avait vu de l'intérieur que le travail n'était pas ce qu'il avait cru. Pas besoin d'une annonce dramatique. Un regard vers l'intérieur suffit.

Les idées des *Recherches* — la signification comme usage, le contexte comme base de la compréhension, la ressemblance de famille entre les concepts — sont devenues le fondement de la linguistique moderne et, indirectement, du traitement du langage naturel. Quand un transformer calcule l'attention entre les tokens, il fait implicitement ce que Wittgenstein avait décrit en mots : il cherche des connexions contextuelles, pas des définitions abstraites. Wittgenstein n'a pas prévu les transformers. Mais les transformers sont, architecturalement, wittgensteiniens. L'idée a survécu à l'autocorrection et est réapparue sous une forme que son auteur n'aurait pu imaginer.

Comment un homme qui pensait avoir fermé toutes les questions devient-il la cause de l'ouverture de nouvelles ? Parce qu'il avait le courage de regarder vers l'intérieur — et de changer de direction.

XI. Attention Is All You Need

En 2017, huit chercheurs de Google publient un article dont le titre était une arrogance qui s'est avérée justifiée : *Attention Is All You Need*. L'architecture transformer — un mécanisme d'attention qui calcule les relations entre tous les tokens simultanément, en parallèle, sans récurrence — était un bond en avant. En 2025, cet article a été cité plus de 173 000 fois, ce qui le place parmi les dix articles les plus cités du XXI^e siècle.

C'était une solution élégante. Et comme toute solution élégante, elle portait en elle le germe de ses propres limites.

Les transformers nécessitent un temps de calcul qui est quadratique par rapport à la taille de la fenêtre de contexte. Chaque token doit être comparé à chaque autre token. Doublez le contexte — quadruplez le calcul. Ce n'est pas un défaut d'implémentation. C'est la nature mathématique du mécanisme d'attention. Le mur est connu depuis le début. Et depuis le début, on sait vers où il mène : des modèles plus grands nécessitent plus de centres de données, plus d'électricité, plus de systèmes de refroidissement. L'industrie a répondu par la mise à l'échelle — plus du même, pas différemment. Des alternatives existent — architectures hybrides, modèles à espace d'état, attention linéaire — mais chaque approche qui fonctionne en temps sous-quadratique perd fondamentalement certaines capacités que le transformer

possède. La complexité quadratique n'est pas un bug qui attend un correctif. Pour certaines tâches, c'est le prix de la compréhension elle-même.

Et pourtant — l'industrie continue de construire des centrales électriques. Parce que c'est le chemin que nous connaissons.

Il y a une habitude qui m'a aidé à ne pas perdre mes repères dans ce domaine : quand un problème d'IA me captive, je cherche des sources d'avant 2017. Je cherche comment on pensait avant le paradigme dominant. Non pas parce que l'ancien est meilleur — mais parce que chaque paradigme a des angles morts par rapport auxquels celui qui l'a précédé n'était pas aveugle. Le regard X-Ray sur le présent exige parfois un passage par le passé.

Dans une réunion quelque part, un collègue plus jeune qui n'a pas encore appris comment "ça ne se fait pas" pourrait demander : pourquoi ne pas changer la définition même de ce que nous demandons au modèle ? Au lieu de mettre à l'échelle un problème quadratique, peut-être que la question n'est pas mal résolue — peut-être qu'elle est mal posée. Wittgenstein était cet enfant pour lui-même. Il n'avait que du papier et la volonté de démolir son propre système. L'industrie de l'IA construit des centrales électriques pour faire plus du même.

Cet enfant n'est pas encore venu. Ou il est venu et il n'y avait personne pour l'entendre.

Quand l'IA se développe exclusivement par la mise à l'échelle, les seuls qui peuvent participer sont ceux qui ont des ressources. Et les ressources ne sont pas réparties équitablement. Les langues parlées par plus de 50 millions de personnes sont classées dans la catégorie "langue à faibles ressources" — non pas parce que peu de gens les parlent, mais parce qu'elles sont absentes de l'espace numérique. Le fine-tuning nécessite du matériel coûteux. Les logiciels écrits pour cela ne démarrent même pas sans GPU. Le savoir enfoui dans les langues que le modèle n'a jamais vues, dans les cultures non représentées dans l'ensemble d'entraînement — pour l'IA, ce savoir n'existe pas.

Toute l'IA est asymétrique par rapport au savoir tiré des sources de l'Occident mondial et, pratiquement parlant, de deux langues dominantes. Ce n'est pas une coïncidence — c'est la conséquence d'un monoculture architecturale qui s'inscrit dans les valeurs et les hypothèses que le modèle acquiert avec ses données.

SIR — mon projet de recherche sur l'IA agentique et le fine-tuning — est né en partie de cette frustration. Une tentative de construire, sans centre de données, sans GPU, avec une attitude X-Ray envers les ressources disponibles, quelque chose qui nécessiterait autrement l'infrastructure du Nord global. Ce n'est pas une déclaration politique. C'est la même attitude que pour le dictionnaire de données : pourquoi ne pas regarder de l'intérieur si le tabou est une impossibilité technique — ou simplement un réflexe institutionnel ?

IX. Synthèse : X-Ray comme système de valeurs

Tout cela — l'art aborigène X-Ray, la rébellion de Strummer, le méta-apprentissage, la boîte noire et le dégradé de gris, l'émergence, l'auto-organisation, l'évolution, la résilience, la neuroplasticité — n'est

pas une collection aléatoire d'idées intéressantes. Ce sont les facettes de la même attitude envers la connaissance et envers la construction.

Illich m'a montré que les institutions peuvent systématiquement empêcher ce qu'elles prétendent produire. Abbott m'a montré qu'une prison peut être faite de cadres de référence, pas de barreaux — et qu'il n'y a pas de sortie tant qu'on ne suppose pas qu'il existe une dimension qu'on ne voit pas. Je n'ai pas lu ces deux livres comme de la littérature académique. Je les ai lus comme des diagnostics.

L'approche X-Ray ne promet pas une compréhension complète. Elle promet une meilleure vue que celle que vous auriez sans elle. Couche par couche, métadonnée par métadonnée, observation par observation — je construis une compréhension de systèmes qui sont intrinsèquement opaques. Et je sais que cette compréhension est toujours partielle. L'ombre de la sphère dans Flatland n'est pas la sphère — mais elle me dit que la sphère existe, qu'elle se déplace, qu'il y a un mouvement que je peux suivre. Ce n'est pas rien. C'est parfois tout ce que nous avons.

L'émergence et l'auto-organisation me rappellent que je n'ai pas besoin de contrôler chaque détail — ce qui compte est de comprendre les règles à partir desquelles le comportement naît. Mais elles me rappellent aussi que la stabilité n'est pas la preuve de la justesse, et que les mêmes forces qui construisent une fourmière peuvent mener une colonie en cercles.

Les algorithmes génétiques me rappellent que la stagnation est plus dangereuse que l'errance. Que la convergence ressemble au succès et l'est parfois — mais que sans mutation je ne pourrai jamais savoir s'il existe une meilleure solution de l'autre côté de la vallée où je me suis confortablement installé.

La résilience me rappelle que la perfection n'est ni atteignable ni nécessaire. Que le seul composant qui a survécu au crash de mon serveur était celui architecturalement isolé de tout ce qui pouvait tomber en panne. Et que cette leçon a coûté 9,5 heures d'indisponibilité à trois heures du matin.

La neuroplasticité me rappelle que la boîte noire la plus difficile est celle que je porte avec moi. Que l'expertise et l'aveuglement ne sont pas des contraires — parfois ce sont les mêmes voies synaptiques. Et que l'attitude X-Ray est, en fin de compte, la capacité de tourner l'appareil vers soi-même — et de regarder.

Le méta-apprentissage est le fil qui traverse tout : je demande toujours non seulement ce que j'apprends, mais comment je l'apprends, et comment je pourrais l'apprendre mieux. Cette question n'est pas un méta-commentaire qui vient à la fin d'un projet. Elle vient au début. Et elle change la façon dont je traverse tout le reste.

...

Regardez les systèmes que vous utilisez chaque jour. Les outils qui ont donné des résultats pendant si longtemps que vous avez cessé de vérifier les prémisses. Les habitudes qui se sont stabilisées au point où vous ne les voyez plus comme des choix. L'expertise qui est devenue un cadre de référence.

Êtes-vous dans un optimum local ? Comment le sauriez-vous ?

La stabilité n'est pas la preuve de la justesse. Un système qui ne tombe jamais en panne est peut-être en parfaite stagnation. Un expert qui ne fait jamais d'erreurs est peut-être une personne qui a cessé d'explorer.

X-Ray n'est pas une réponse. C'est l'attitude avec laquelle nous abordons les questions.

...

Chaque projet est une expérience. Chaque expérience est un point de données. Chaque point de données est un pixel dans l'image X-Ray de ce que je construis.

Ce document est vivant.

Il n'est pas terminé au moment de l'écriture — il sera complété par de nouvelles expériences, de nouveaux projets, de nouvelles perspectives. Places pour une expansion future :

→ *Briscola et DQN — l'observabilité partielle comme problème X-Ray : un agent qui apprend à inférer ce qu'il ne voit pas, construisant un modèle à partir des traces disponibles.*

→ *Katalog comme archive vivante — un méta-registre qui éclaire tous les autres projets de l'intérieur, une mise en œuvre de la philosophie X-Ray dans une base de données.*

→ *Les modes de défaillance comme philosophie — documenter non seulement ce qui a fonctionné, mais ce qui a échoué et comment le système s'en est remis. C'est la documentation X-Ray.*

→ *L'informatique quantique et les automates cellulaires — un nouveau terrain pour les mêmes questions : qu'y a-t-il à l'intérieur, quelles sont les règles minimales, comment la complexité naît-elle.*

Flavio · 2026 · Document vivant · v3b

Annexe

Note sur le laboratoire

Tous les exemples qui suivent sont nés de mon bac à sable personnel — un environnement isolé dans lequel je développe des projets, teste des idées et apprends de mes erreurs. Les conséquences des erreurs et des crashes restent dans cet environnement et n'affectent pas le monde extérieur, sauf parfois mon sommeil quand ils surviennent à trois heures du matin.

Projet James : l'allumette dans le chambranle

Dans les films de James Bond, il y a une scène qui se répète : avant de quitter sa chambre d'hôtel, Bond cale discrètement une allumette dans le chambranle de la porte. À son retour, il vérifie si l'allumette est toujours là. Si elle n'y est pas — un ennemi est passé, ou est encore à l'intérieur.

Même principe, substrat différent.

James est une application sur un appareil mobile. L'appareil peut être accroché à une poignée de porte, placé dans le tiroir d'une table de nuit, appuyé contre une fenêtre ou caché dans une mallette contenant des documents importants. Il surveille l'accélération linéaire — un signal dont la gravité a déjà été retirée, de sorte qu'il réagit au toucher plutôt qu'aux vibrations du bâtiment. Quand quelqu'un ou quelque chose déplace l'objet, une notification ntgy parvient au téléphone du propriétaire en quelques secondes.

Pas de caméra. Pas de services cloud. Pas de matériel spécialisé. Tout ce qui existait déjà dans l'écosystème — redéployé dans une nouvelle fonction.

Le regard X-Ray sur James n'est pas technique — c'est une attitude. L'allumette de Bond dans le chambranle n'était pas un système de sécurité. C'était un indicateur d'état. James fait la même chose : il ne protège pas, il éclaire. Il vous dit ce qui s'est passé pendant votre absence.

Serveur Balsam : quand le noyau tue un processus

Dans la nuit du 23 au 24 mars 2026, mon serveur Balsam s'est figé et a cessé de répondre. La connexion SSH a coupé. La console VNC était inactive. Le serveur est resté indisponible pendant 9,5 heures, de 03h00 à 12h30.

La cause : le OOM Killer de Linux — un mécanisme du noyau qui, quand le système manque de mémoire libre, trouve et tue le processus qui consomme le plus de RAM — a tué le conteneur Docker d'Ollama, qui occupait environ 30 % de la RAM totale disponible. Sur un système sans partition swap, la seule issue était SIGKILL. Le serveur s'est figé immédiatement après.

L'hypothèse préalable était simple : trois services peuvent coexister sur la RAM disponible sans limites explicites. Personne n'avait pris en compte la possibilité qu'au pic de consommation, ils puissent simultanément demander plus que ce qui était disponible, et que sans swap, le noyau n'a pas d'autre solution que de tuer.

La panne n'a pas été détectée en temps réel. Je l'ai découverte le matin quand SSH ne se connectait pas. L'analyse post-mortem via dmesg — un outil qui lit le journal d'événements du noyau du système d'exploitation — a confirmé l'événement OOM avec l'identifiant exact du processus et la quantité de mémoire consommée.

La récupération n'a été rapide que grâce à un seul composant qui fonctionnait de manière autonome et indépendamment de tout le reste : la sauvegarde rsync de 03h01 cette même nuit était complète. Sans perte. La sauvegarde était la seule chose dans toute l'infrastructure architecturalement isolée du composant dont elle était censée survivre à la défaillance.

Immédiatement après le redémarrage : un fichier swap de 50 % de la RAM totale et une limite mémoire Docker sur le conteneur Ollama — un plafond strict de 33 % de RAM plus 12 % de swap. Les deux survivent à un redémarrage.

La leçon que je n'avais pas avant cette nuit : concevoir pour la récupération signifie modéliser explicitement ce qui se passe quand l'hypothèse de fonctionnement normal ne tient pas. La différence n'est pas dans la complexité — dans ce cas c'était une commande pour le swap et un paramètre Docker. La différence est dans l'attitude.

Katalog : quand l'extension ment

Il existe aussi un type de panne plus silencieux — pas de crash serveur, pas de OOM Killer. Dans le projet Katalog, le système a tenté de lire un ensemble de fichiers .docx en utilisant des méthodes standard : décompression de l'archive ZIP et conversion via pandoc. Les deux outils ont échoué avec une erreur. Les fichiers portant l'extension .docx étaient en réalité de simples fichiers texte Markdown en UTF-8 — pas d'enveloppe ZIP, pas de XML à l'intérieur.

La panne n'était pas visible pour l'utilisateur. Elle était cachée dans une erreur d'un outil appelé en arrière-plan. Sans interception explicite des exceptions et stratégie de repli — ouvrir le fichier comme texte brut quand la décompression échoue — le système aurait silencieusement ignoré le fichier ou retourné un résultat vide.

Une extension de fichier n'est pas un contrat. Elle décrit une intention, elle ne garantit pas le contenu. Toute interface avec des données externes a besoin d'une stratégie de repli — explicite, visible et de valeur égale au chemin principal.

Le même projet a révélé un autre paradoxe, cette fois dans ma propre documentation. Pendant des années, j'ai documenté mes projets en format .docx — le standard de facto pour la distribution de documents, un format que chaque bureau et chaque client peut ouvrir. Une habitude acquise par socialisation professionnelle, tracée au point de l'invisibilité.

.docx est, dans le monde des formats de documentation, l'une des boîtes noires les plus opaques. Une archive ZIP contenant des fichiers XML, des déclarations d'espaces de noms, des fichiers de relations — tout cela juste pour stocker du texte avec un titre et des italiques. Markdown est l'opposé : un fichier texte brut avec une poignée de caractères de syntaxe. Il se lit dans n'importe quel éditeur, il est cherchable, il est versionnable, il survit à tout.

La solution n'était pas l'un ou l'autre. Je documente maintenant en format Markdown — et quand j'ai besoin de distribuer, je génère un .docx. Le même contenu, le bon format pour le bon contexte.

Et ce changement m'a, de manière inattendue, rapproché de l'attitude X-Ray : la source de vérité est désormais lisible sans outil spécial. De l'intérieur.